



Course Project - Final Report

Near Real-Time Financial Forecasting on AWS

A self-running, two-horizon, serverless forecasting pipeline for TSLA / MSFT / NVDA

Table of Contents

1	Executive Summary	1
2	What the Project Finally Does	1
3	Scope: First Draft vs. Final Delivery	2
4	Methodology: How We Organised and Worked	3
4.1	Division of responsibilities	3
4.2	What we agreed before writing any code	4
4.3	Handoffs: explicit interface contracts	4
4.4	Order of work and working environments	4
4.5	Communication and tools	5
5	System Architecture	5
5.1	Multi-account topology	5
5.2	End-to-end data flow	6
5.3	The orchestrated hourly cycle	7
6	Component-by-Component Implementation	8
6.1	Data ingestion - Ilias	8
6.2	Sentiment scoring - Ralph	9
6.3	Data preparation (ETL) - Dalibor	9
6.4	Forecasting - Ralph	10
6.5	Storage: DynamoDB and S3	11
6.6	Anomaly alerting - Arthur	11
6.7	Orchestration - Step Functions	12
6.8	Monitoring - CloudWatch and the freshness probe	12
6.9	Self-improvement - weekly retraining	13
7	The Strategic (Daily) Horizon - a Second Model	13
8	Visualisation: the QuickSight Dashboard - Francesco	14
8.1	Hourly Forecasts sheet (operational)	14

8.2	7-day Daily Forecasts sheet (strategic)	16
8.3	Interactive features	17
9	Services and Resources: Benefits and Alternatives	17
10	Twelve-Factor Methodology	18
11	Code Organisation and Engineering Practices	20
11.1	Repository structure	20
11.2	From the labs to the project	20
11.3	Docker and CI	21
12	Problems Encountered and Solutions	21
12.1	Coding and data-contract problems	21
12.2	Services and cross-account problems	22
12.3	Team-organisation problems	22
12.4	What the monitoring layer caught	23
13	Results and Live Accuracy	24
14	Cost Analysis	25
15	Hours Invested and Deviation	25
16	Conclusions and Future Work	27
Annex A	- Additional Evidence	27
Annex B	- Reproducibility	28

Group 11_02 - authors and primary responsibility:

Member	Pipeline stage	Core AWS services
Ilias El Mariky	Data ingestion (1-2)	Lambda, EventBridge
Dalibor Švonavec	Data preparation / ETL (3-4)	SQS, AWS Glue, S3
Ralph Khairallah	Sentiment + forecasting (5-6)	Comprehend, SageMaker, DynamoDB
Francesco Barillari	Visualisation (7-8)	Athena, QuickSight
Arthur Bohin	Alerting + orchestration (9-10)	SNS, Step Functions, CloudWatch

Repository: `github.com/Francesco-Barillari/ccbda-project`
Region: `eu-west-1`
Date: May 26, 2026

1 Executive Summary

At a glance

Project 11_02 is a **self-running, near real-time financial forecasting pipeline** built entirely on AWS. Every hour it ingests live stock prices and financial news for three tickers (**TSLA, MSFT, NVDA**), scores news sentiment with Amazon Comprehend, forecasts the next hour of prices with a DeepAR model on a *serverless* SageMaker endpoint, refreshes a QuickSight dashboard, and emails the team whenever an actual price falls outside the predicted confidence band. A **second, strategic model** forecasts the daily close 7 days ahead. After deployment there is **no human in the loop**: EventBridge crons and Step Functions state machines drive the whole system. The work spans **four AWS accounts** wired together with S3, DynamoDB and SNS resource policies, and follows the **twelve-factor methodology** (10/12 fully satisfied) plus the CI/CD, observability, serverless and log-analysis patterns from the course labs.

Headline fact	Value
End-to-end automation	fully automatic; no manual step after deploy
Forecast horizons	2 (hourly: 1 h ahead + daily: 7 days ahead)
Live hourly accuracy (MAPE)	TSLA 0.19%, MSFT 0.13%, NVDA 0.35%
Anomaly rate (market hours)	0% across all three tickers
AWS accounts integrated	4 (cross-account IAM via resource policies)
Lambdas in production	11 (+ 1 Glue job, 2 serverless endpoints)
Storage layer	DynamoDB (on-demand) for forecasts + S3 (Parquet) data lake
Twelve-factor compliance	10 full, 1 N/A by design, 1 partial
Cumulative AWS spend	≈ \$0.35 of a \$100 academic credit
Steady-state operating cost	≈ \$0.20/month (Glue + S3; compute scales to zero)

Table 1: Headline numbers for the delivered system.

2 What the Project Finally Does

The delivered system is a **continuously-running data product**, not a one-off script or a static notebook. It demonstrates the cloud-native principles of the course - decoupling, elasticity, design-for-failure and observability - on a realistic, end-to-end problem: turning a live financial data feed into actionable forecasts and alerts.

Concretely, once deployed the system performs the following loop forever, with no operator intervention:

1. **Ingest.** A scheduled Lambda polls the Alpha Vantage API for the latest price quote and news headlines of the same three tickers (TSLA, MSFT, NVDA), then routes the prices to one Amazon SQS queue and the news to a separate one (each feed travels on its own queue). Nine API keys are rotated by the hour to stay inside the free-tier rate limit.
2. **Score sentiment.** An SQS-triggered Lambda sends every news headline to Amazon Comprehend's `DetectSentiment`, converts the class probabilities into a single score in $[-1, +1]$, and persists it to S3.
3. **Prepare data.** An hourly AWS Glue job drains the prices queue, rounds timestamps to a clean 5-minute grid, gap-fills missing ticks, joins the latest sentiment using a backward as-of merge with a 4-hour tolerance, and writes one gapless Parquet file per ticker - the exact shape DeepAR needs.
4. **Forecast.** A Step Functions state machine fires a forecast Lambda that backfills the actual

outcomes of previous predictions, then calls the serverless DeepAR endpoint with the last 10 hours of context plus sentiment as a related time series, obtaining $p_{10}/p_{50}/p_{90}$ bands for the next hour. Predictions are written to DynamoDB and to S3.

5. **Visualise.** A refresh Lambda rewrites the Athena-backed CSVs; a live QuickSight dashboard (Direct Query) shows actuals against the prediction band over two interactive sheets (hourly and daily).
6. **Alert.** A DynamoDB Stream notifies a cross-account Lambda in another teammate's account whenever a backfilled actual lands outside its predicted band during market hours; it publishes a formatted alert email through SNS.
7. **Self-heal and self-improve.** CloudWatch alarms watch every critical Lambda, the state machine, and a custom data-freshness metric; two weekly retraining Lambdas refresh both DeepAR models so accuracy does not drift.

Because an **always-working demo** was a core requirement, the dashboard is permanently live and the pipeline has been running unattended for days, accumulating a real prediction-versus-actual history that we use throughout this report as evidence rather than as illustration.

3 Scope: First Draft vs. Final Delivery

The first-draft document proposed a near real-time forecasting pipeline using essentially the AWS service set we ultimately used. The architecture survived contact with reality remarkably well - every service named in the draft appears in the final system. The differences are in *which assets we track, what we added beyond the plan, and what we consciously dropped*.

Aspect	First draft (plan)	Final delivery (built)
Assets	AAPL, TSLA + crypto (BTC, ETH); "stocks vs. crypto" comparison	TSLA, MSFT, NVDA (three equities); crypto dropped
Forecast horizon	A single near real-time horizon	Two horizons: hourly (1h) <i>and</i> daily (7d), two separate DeepAR models
Forecasting service	Amazon Forecast (managed)	SageMaker DeepAR on serverless endpoints; switched after Amazon Forecast was retired for new projects
Retraining	"hourly or daily depending on cost"	Weekly automated retrain Lambdas for both models
Orchestration	Step Functions planned	Step Functions delivered for <i>both</i> the hourly and daily chains
Sentiment	Sentiment as related time series	Delivered as DeepAR <code>dynamic_feat</code> , with a 4h decay so stale news fades
Monitoring	CloudWatch (logs/errors)	CloudWatch + a custom data-freshness metric and 11 alarms
Engineering	Not detailed	Added CI/CD (GitHub Actions), Docker , pytest suite, twelve-factor audit
Alerts	Email <i>and</i> SMS	Email only (SMS dropped - see below)

Table 2: Scope evolution from the first-draft proposal to the delivered system.

Features added or changed beyond the draft. The most significant addition is the *second forecasting horizon*: a 1-hour forecast is operationally interesting but does not match how a real trading or risk team consumes forecasts (days-to-weeks ahead), so rather than stretch the 5-minute

model - which has almost no skill past ~ 30 minutes - we built a **separate daily-close model** on years of daily history (Section 7). We also changed the forecasting service itself: the first draft planned **Amazon Forecast**, but it was retired for new projects, so we moved to **SageMaker DeepAR**, which also gave us probabilistic $p_{10}/p_{50}/p_{90}$ bands and a single global model across tickers. Finally we added weekly automated retraining, a data-freshness alarm that catches silent ingestion failures, and the full lab-derived engineering layer (CI, Docker, tests).

Features deliberately not implemented:

- *Crypto and the stocks-vs-crypto comparison*: Alpha Vantage's news endpoint does not return per-asset sentiment for crypto tickers, and sentiment is a core model input, so crypto could not be fed through the pipeline consistently. We narrowed to three correlated US equities, which also keeps the global DeepAR model on a coherent volatility regime (mixing crypto and equities widened the prediction bands).
- *SMS alerts*: email is free and reliable, while SMS costs $\approx \$0.06/\text{message}$ and adds an SNS sandbox-verification burden for a student account, so it was not worth it for a demo.
- *A separate staging environment*: we ran a single production environment (the dev/prod-parity trade-off discussed in Section 10).

All three omissions were scope decisions, not failures, and each is justified where it appears.

4 Methodology: How We Organised and Worked

The brief asks explicitly for the *methodology* used to build the project: how responsibilities were divided, how meetings and idea exchange were organised, which working environments and tools we used. Because the five of us are exchange/master's students from different home universities, working across *different AWS accounts*, coordination was as much an engineering problem as the code itself. We treated it that way from day one and wrote it down.

4.1 Division of responsibilities

We split the pipeline into five contiguous stages, one owner each, so that every person owned a self-contained slice with a clear input and a clear output. The split mirrors the natural data flow of the system.

Member	Steps	Responsibility	Services
Ilias El Mariky	1-2	Fetch prices + news from external APIs into queues	EventBridge, Lambda
Dalibor Švonavec	3-4	Clean / transform raw messages into model-ready data	SQS, Glue, S3
Ralph Khairallah	5-6	Sentiment scoring + forecasting + model lifecycle	Comprehend, SageMaker
Francesco Barillari	7-8	Storage query layer + interactive dash-board	DynamoDB, Athena, QuickSight
Arthur Bohin	9-10	Alerting and end-to-end orchestration / monitoring	SNS, Step Functions, CloudWatch

Table 3: Task split agreed in the *Team Coordination Plan* (committed to the repository at docs/team/).

The stages form a strict dependency chain - each person consumes the previous person's output - with one person (Arthur) wiring the whole thing together at the end and consuming the anomaly signal:

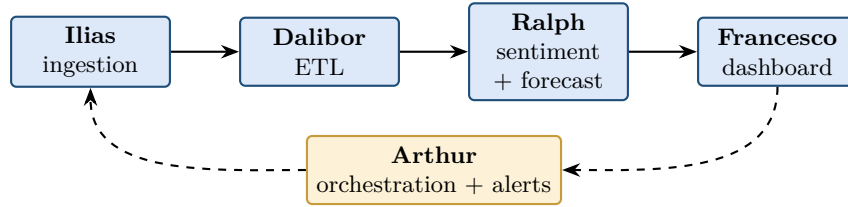


Figure 1: Dependency flow. Ilias is the source; Dalibor, Ralph and Francesco each build on the previous output (and could start with mock data in parallel); Arthur orchestrates the full chain with Step Functions and consumes the anomaly flag.

4.2 What we agreed before writing any code

The single most useful thing we did was to freeze a small set of shared conventions *before* anyone started coding, and commit to not changing them silently. This is what let five people on five machines and four AWS accounts converge instead of diverge:

- **Region:** eu-west-1 for every resource (Dalibor’s bucket lives there).
- **Naming prefix:** proj1102- on everything we create, so resources are greppable across accounts.
- **Tickers, timestamp format** (ISO-8601 UTC), **S3 bucket name and paths, message format** between Ilias and Dalibor, **dataset format** between Dalibor and Ralph, **DynamoDB schema** between Ralph and Francesco, and **retrain cadence**.

These conventions were captured in the *Team Coordination Plan* and a shared resource tracker (a Google Doc table of *resource* → *name* → *owner* → *ARN/URL*) so nobody had to dig through someone else’s AWS console to find a queue URL or a table ARN.

4.3 Handoffs: explicit interface contracts

Because each stage is owned by a different person in a different account, the *interface between two adjacent stages* is where things break. We therefore wrote a short **handoff document for every adjacent pair** - who gives what to whom, with a concrete worked example - and committed all four to docs/team/. They behave like API contracts.

Handoff	Contract (the interface that must stay stable)
Ilias → Dalibor	SQS queue names + ARNs, and the JSON shape of price / news messages.
Dalibor → Ralph	S3 bucket + path of the clean data, file format (Parquet) and column names.
Ralph → Francesco	S3 path of forecast results + the DynamoDB table name and schema (ticker, timestamp, p10/p50/p90, is_anomaly).
Francesco → Arthur	The DynamoDB anomaly-flag format (so the alert Lambda can read it) + the dashboard URL.
Everyone → Arthur	The ARN of every Lambda, Glue job and endpoint, and every log-group name - Arthur needs them for Step Functions and CloudWatch.

Table 4: Pairwise handoff contracts. Several of the issues in Section 12 are exactly the cases where the real implementation drifted from one of these contracts.

4.4 Order of work and working environments

We sequenced the four weeks so that nobody was blocked waiting for an upstream component that did not exist yet:

- **Week 1 - build in parallel.** Agree the conventions above, then each member builds their own component against *mock data* so nobody waits on an upstream piece (Dalibor wrote a script that pushed dummy messages to SQS, giving the downstream stages input before Ilias’s live feed

existed).

- **Week 2 - connect adjacent pairs.** Replace the mock data with real handoffs, one boundary at a time: Ilias→Dalibor (messages), Dalibor→Ralph (clean data), Ralph→Francesco (forecasts + anomalies), Francesco→Arthur (anomaly→SNS).
- **Week 3 - orchestrate.** Arthur wires the full pipeline into Step Functions and brings up the CloudWatch dashboard and alarms.
- **Week 4 - harden and document.** Run the system unattended for ≥ 24 h, then write the report and prepare the demo.

On **working environments** (dev / staging / production): because every resource name is read from configuration (twelve-factor factor 3), each developer effectively had a *development* workspace - mock data, local invocation, a personal account or prefix - before promoting code to the single shared *production* pipeline. We did not stand up a separate always-on staging environment; for a one-semester student project the cost of operating two parallel pipelines was not justified (discussed honestly under factor 10, Section 10).

4.5 Communication and tools

Channel / tool	Use
WhatsApp group chat	Daily asynchronous updates, quick unblocking, “I just changed X” notifications.
Weekly team meeting + ad-hoc calls	One scheduled sync per week, plus many short calls between adjacent pairs (e.g. Ilias↔Dalibor on the message schema, Ralph↔Francesco on the DynamoDB schema) - the dependency chain meant most co-ordination was pairwise.
Shared Google Doc	Living resource tracker (names, ARNs, URLs) + the report-preparation document each member filled for their section.
GitHub (single repo)	One private monorepo, Francesco-Barillari/ccbda-project , versioning every component; pull requests for the riskier merges (e.g. the sentiment as-of-join fix).
Handoff + coordination PDFs	The written contracts in docs/team/ , kept in the repo so the rules live next to the code.

Table 5: Communication channels and collaboration tools.

The golden rule, written at the top of the coordination plan, was: *any change in schema or naming → tell the group immediately*. Most of the integration bugs we did hit (Section 12) trace back to a moment where that rule was not followed and a producer changed something a consumer silently depended on.

5 System Architecture

5.1 Multi-account topology

A defining constraint shaped the whole architecture: the shared team *sandbox* account blocks Amazon Comprehend at the account level and its scoped-down deployer user cannot manage SageMaker, Glue, or cross-account stream subscriptions. Rather than fight the sandbox, **each contributor runs their own compute in their own AWS account**, and the team bucket `s3://proj1102-data` is the shared data plane. The cost is more cross-account IAM plumbing; the benefit is that each person moves independently without waiting on a team-account permission change.

Account	ID	Hosts
Ralph (personal)	148557232117	Sentiment, forecast & daily Lambdas, both DeepAR endpoints, DynamoDB tables, Quick-Sight, Athena, retrain + freshness Lambdas, Step Functions, infra-alarms SNS
Team sandbox	834922934600	SQS queues, Glue job, glue-trigger Lambda, proj1102-data S3 bucket
Arthur (personal)	162655485124	Anomaly-alert Lambda, proj1102-alerts SNS topic
Ilias (personal)	628454062331	Data-ingestion Lambda + EventBridge schedule

Table 6: Four-account topology. Cross-account access is granted by S3 bucket policies, two DynamoDB resource policies (table + stream), and an SNS topic policy.

5.2 End-to-end data flow

Figure 2 shows the operational (hourly) pipeline. Nodes are coloured by the account that owns them. The system is strictly event-driven and decoupled: SQS buffers ingestion from processing, S3/Parquet decouples ETL from forecasting, and DynamoDB Streams decouples forecasting from alerting.

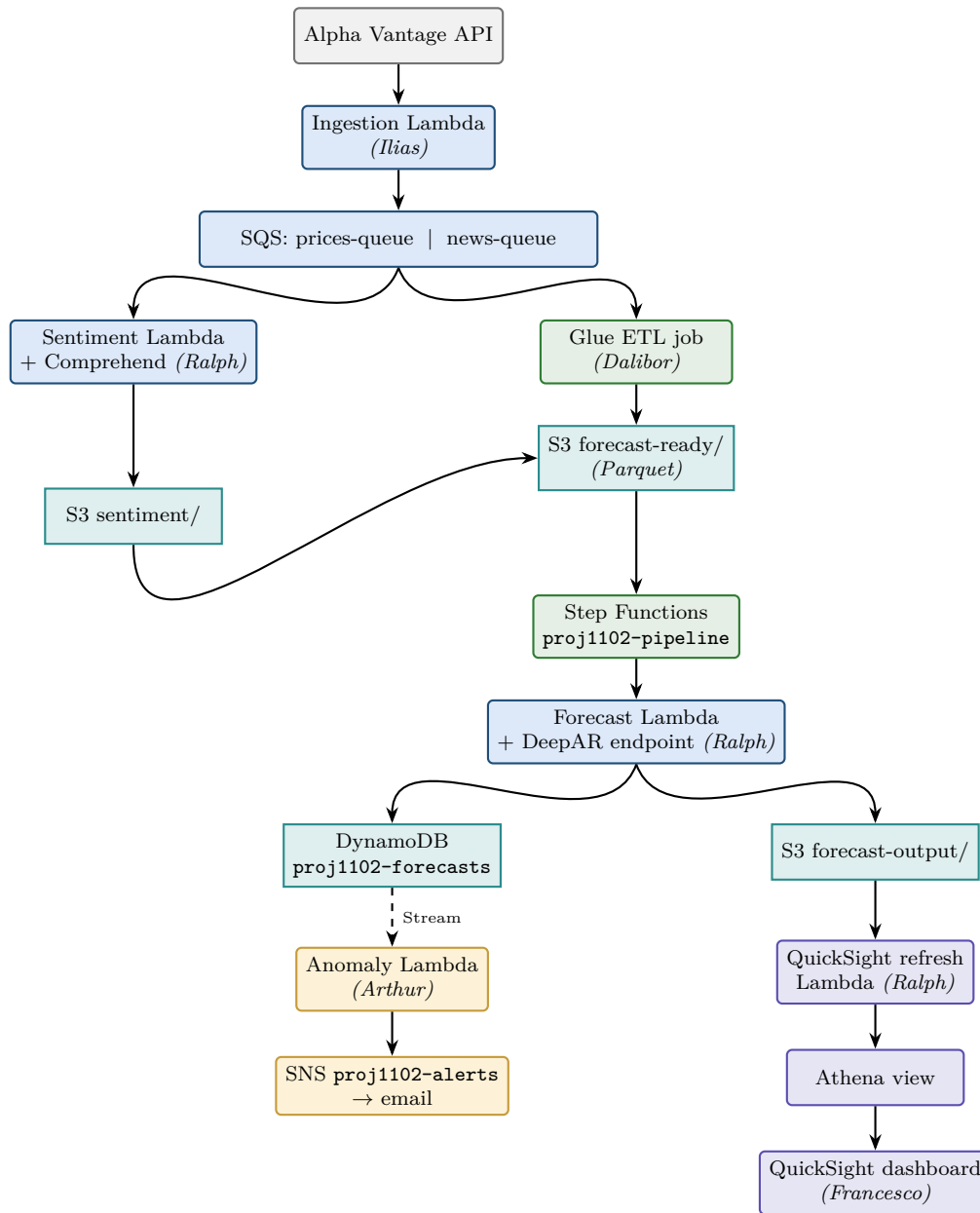


Figure 2: Operational (hourly) architecture, coloured by owning account: ■ Ralph/compute, ■ team account (Glue / Step Functions), ■ Arthur (alerts), ■ Francesco (visualisation), ■ storage. Buffers (SQS, S3, DynamoDB Streams) keep every stage decoupled.

5.3 The orchestrated hourly cycle

Each clock hour the components fire in a fixed, dependency-respecting order. Scheduling the consumer strictly *after* the producer is what keeps predictions always ahead of actuals (the fix for issue 12.1.1).

When (UTC)	What happens
:00-:30	Glue runs in the team account → writes <code>forecast-ready/<ticker>.parquet</code> .
:35	Freshness probe Lambda → publishes the max parquet-age CloudWatch metric.
:45	Step Functions pipeline: Forecast Lambda → DynamoDB + forecast-output CSV, then QuickSight refresh → the Athena CSVs. Any error → SNS <code>proj1102-infra-alarms</code> .
continuous	Sentiment Lambda (SQS-triggered) → <code>sentiment/<ticker>.csv</code> .
continuous	Anomaly Lambda (DynamoDB Stream) → SNS <code>proj1102-alerts</code> (email).
Sun 06:00 / 07:00	Retrain Lambdas → refreshed hourly and daily DeepAR models.

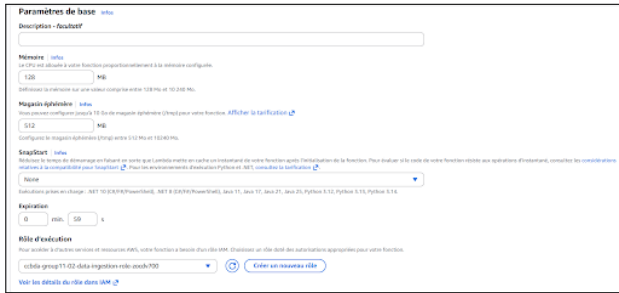
Table 7: The orchestrated hourly cycle - a fixed, dependency-respecting order, with each consumer scheduled strictly after its producer.

6 Component-by-Component Implementation

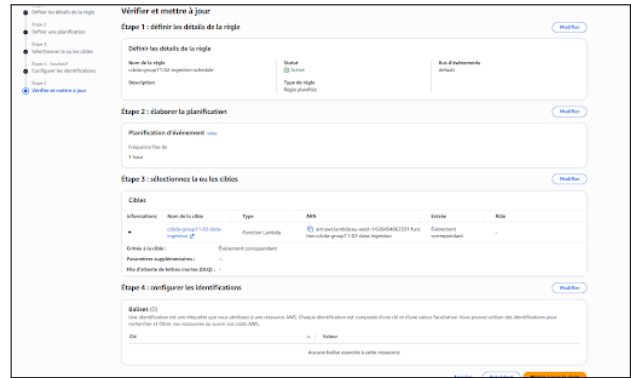
This section walks the operational pipeline stage by stage, in data-flow order. Each subsection states what the component does, the key engineering decisions, and where its source lives in the repository.

6.1 Data ingestion - Ilias

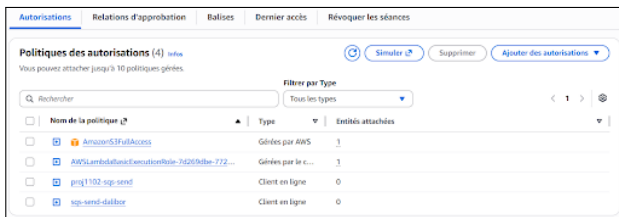
A single scheduled Lambda fetches the latest price (`GLOBAL_QUOTE`) and news (`NEWS_SENTIMENT`) for the three tickers from the Alpha Vantage API and publishes each as a message onto `proj1102-prices-queue` and `proj1102-news-queue`. The free Alpha Vantage tier allows only 25 requests/day per key, so the Lambda **rotates nine API keys** (three per ticker) deterministically by the UTC hour, all kept as Lambda environment variables, to stay within quota; it also pauses 1.2s between calls to respect the one-request-per-second limit. The function runs in Ilias's personal account on an hourly EventBridge schedule and is the single source of live data - until it pushes, the rest of the pipeline has no input, which is exactly why it sits at the head of the dependency chain (Figure 1). Source: `lambda/data_ingestion/`.



(a) Lambda general configuration (memory, execution role).



(b) The EventBridge rule: a scheduled hourly trigger targeting the ingestion Lambda.



(c) Role permissions, including the SQS-send policies for the price and news queues.

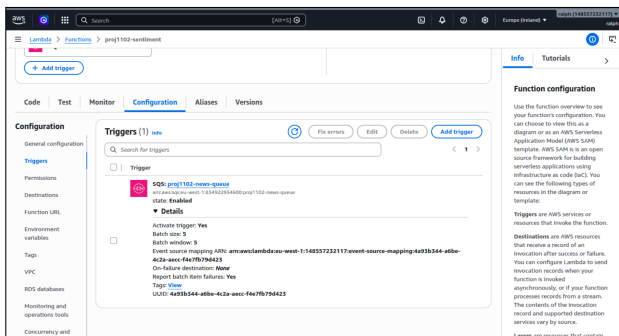


(d) One successful run in CloudWatch: messages sent to the price and news queues.

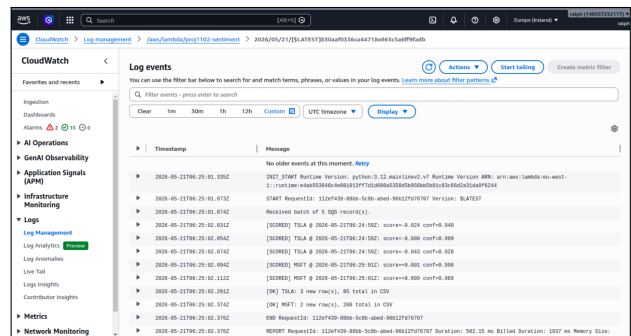
Figure 3: Data ingestion (Ilias's account 628454062331): the hourly EventBridge-triggered Lambda, its role, and a successful run.

6.2 Sentiment scoring - Ralph

An SQS event-source mapping triggers a Lambda for every message on the news queue. Each headline is sent to Amazon Comprehend's `DetectSentiment`, and the per-class probabilities are reduced to a single score $s = P(\text{positive}) - P(\text{negative}) \in [-1, +1]$. The score is appended (read-modify-write) to `s3://proj1102-data/sentiment/<TICKER>.csv` - a *cross-account* write into the team bucket. Comprehend was chosen over a bundled NLP model because it is a managed, pay-per-use service that is free under 10M characters/month and needs no model packaging in the Lambda zip. Source: `lambda/sentiment/`.



(a) The Lambda's SQS trigger on `proj1102-news-queue` (Enabled).



(b) CloudWatch logs: one [SCORED] line per headline (ticker, score, confidence).

Figure 4: Sentiment scoring: the SQS trigger that drives the Lambda, and the [SCORED] log lines showing headlines scored end-to-end.

6.3 Data preparation (ETL) - Dalibor

Dalibor's AWS Glue *Python Shell* job, `team01-data-preparation`, is the heart of the data plane. It began as a proof of concept that read raw data from S3 and wrote CSV; after agreement with Ilias

it switched to **SQS** (no need to persist unprocessed data, and faster), and the output format moved to **Parquet** because it works better with DeepAR. The job is triggered hourly by a thin glue-trigger Lambda on EventBridge. Its final six-step flow turns sparse, irregular SQS messages into the dense, gapless time series DeepAR requires:

1. **Parse & group** SQS messages by ticker; malformed messages are left in the queue for retry (and ultimately a dead-letter queue).
2. **Round & deduplicate**: every timestamp is rounded to the nearest 5-minute slot; duplicates for the same (ticker, timestamp) are dropped, keeping the last value.
3. **Build a complete 5-minute grid** from the anchor timestamp to the latest message, so the output is always a gapless series.
4. **Gap-fill prices**: real rows are left-joined onto the grid; slots with no tick are forward-filled from the last known value.
5. **Join sentiment** with `merge_asof` (backward, 4-hour tolerance): each price row inherits the most recent sentiment score, but stale news older than 4 h is *not* carried forward - a deliberate choice so yesterday's news cannot drive today's forecast.
6. **Concat, dedup, write**: the new portion is appended to the existing history, deduplicated by (ticker, timestamp), and written back as a single Snappy-compressed Parquet file via PyArrow.

A CloudWatch alarm on the Glue job publishes to SNS on any failure in the last 5 minutes, and every SQS queue has a dead-letter queue for poison messages. Source: `glue/data_preparation/job.py`.

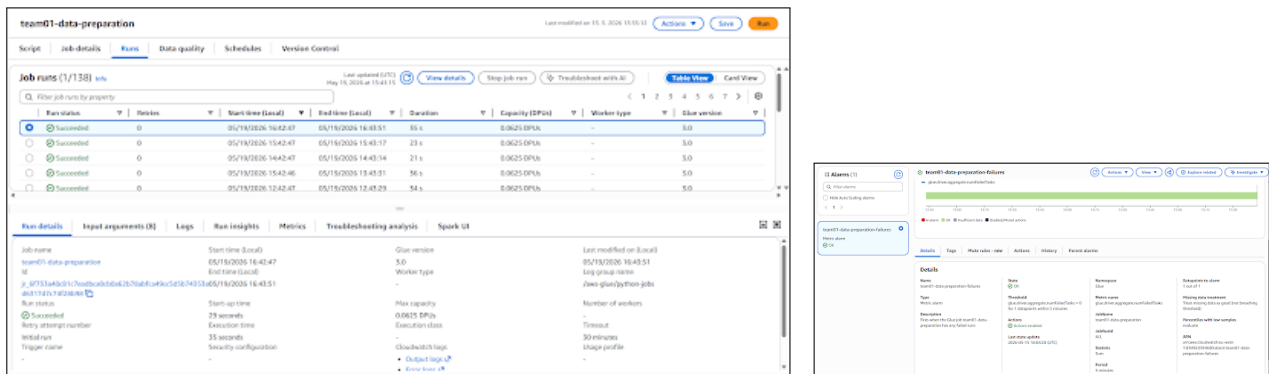


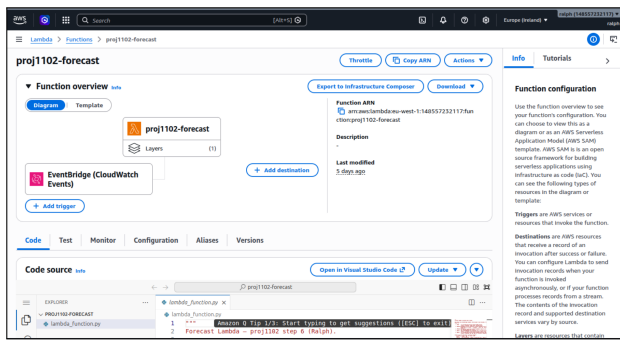
Figure 5: Dalibor's Glue data-preparation job in the AWS console: hundreds of clean hourly runs and the failure alarm wired to SNS.

6.4 Forecasting - Ralph

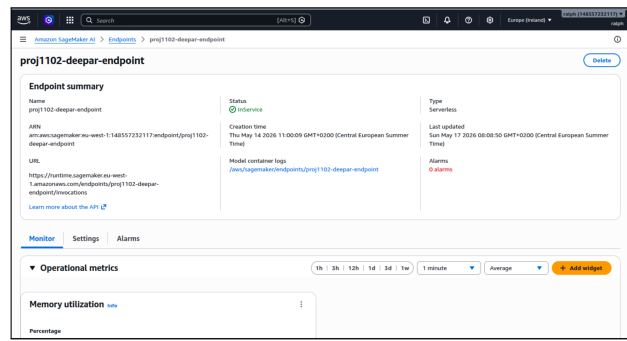
The forecast Lambda is invoked hourly by Step Functions. For each ticker it:

1. reads the latest `forecast-ready/<TICKER>.parquet`;
2. **backfills** the **actual** outcome, **is_anomaly**, **delta** and **severity** onto previously-written DynamoDB rows whose predicted timestamps have now elapsed;
3. calls the DeepAR `serverless` endpoint with the last 120 price points (10h of context) *and* **sentiment_score** as a **dynamic_feat** related time series, receiving $p_{10}/p_{50}/p_{90}$ for the next 12 five-minute steps (1h);
4. writes the new prediction rows to DynamoDB;
5. writes a uniquely-named CSV `predictions_<RUN_TS>.csv` to S3.

The anomaly rule, computed during backfill, is `actual < p10 OR actual > p90`, but only during US regular trading hours (the fix for the closed-market storm, issue 12.1.2). Source: `lambda/forecast/`.



(a) The proj1102-forecast Lambda (Step Functions / EventBridge driven).



(b) The DeepAR endpoint: Status InService, Type Serverless.

Timestamp	Message
	No older events at this moment. Retry
2026-05-21T06:45:19.943Z	INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:eu-west-1::runtime:e4ab53846c4e081013ff7d1d608a535805b956bb5b81c83c66d2a31da8f6244
2026-05-21T06:45:22.389Z	START RequestId: 941c9961-b0f6-4d5d-b0f4-12ec4c85f4fd Version: \$LATEST
2026-05-21T06:45:29.604Z	[OK] TSLA: backfilled 12, wrote 12 predictions, P50 416.88 -> 417.11
2026-05-21T06:45:30.544Z	[OK] MSFT: backfilled 12, wrote 12 predictions, P50 420.58 -> 420.79
2026-05-21T06:45:31.205Z	[OK] NVDA: backfilled 12, wrote 12 predictions, P50 224.22 -> 224.00
2026-05-21T06:45:31.207Z	END RequestId: 941c9961-b0f6-4d5d-b0f4-12ec4c85f4fd
2026-05-21T06:45:31.207Z	REPORT RequestId: 941c9961-b0f6-4d5d-b0f4-12ec4c85f4fd Duration: 8817.92 ms Billed Duration: 11261 ms Memory Size: 512 MB Max Memory Used: 217 MB Init Duration: 2442.98 ms
	No newer events at this moment. Auto retry paused . Resume

(c) CloudWatch: each ticker logs [OK] ...: backfilled 12, wrote 12 predictions.

Figure 6: Forecasting: the Lambda, the serverless DeepAR endpoint it calls, and the per-ticker [OK] log confirming predictions are written every cycle.

6.5 Storage: DynamoDB and S3

Predictions are written in **two passes** that exploit DynamoDB Streams. On **INSERT** (when the prediction is made) the row holds $p_{10}/p_{50}/p_{90}$ with the evaluation fields NULL; on **MODIFY** (the next cycle, once the predicted time has elapsed) the same row gains **actual** and the derived fields. The S3 forecast CSVs accumulate the raw predictions for the dashboard; DynamoDB (proj1102-forecasts, partition **ticker**, sort **timestamp**, PAY_PER_REQUEST, Streams on **NEW_IMAGE**) holds the full prediction+evaluation record forever and is the source of truth for accuracy. Two separate resource policies are attached - one on the table, one on the stream - because AWS rejects stream actions inside a table policy (issue 12.2.1).

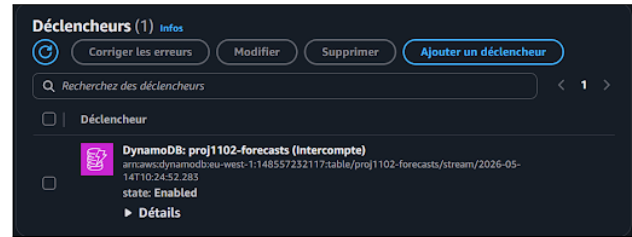
6.6 Anomaly alerting - Arthur

Arthur's Lambda proj1102-anomaly-alert runs in his *own* account and subscribes to the DynamoDB Stream on Ralph's proj1102-forecasts table. It fires whenever the forecast Lambda writes a row, filters on the **is_anomaly** flag (which only becomes true on the backfill pass, once the actual price is known), and on a true value it extracts the ticker, timestamp, predicted range, actual price, delta and severity and publishes a formatted alert to the proj1102-alerts SNS topic, which fans out to the team by email.

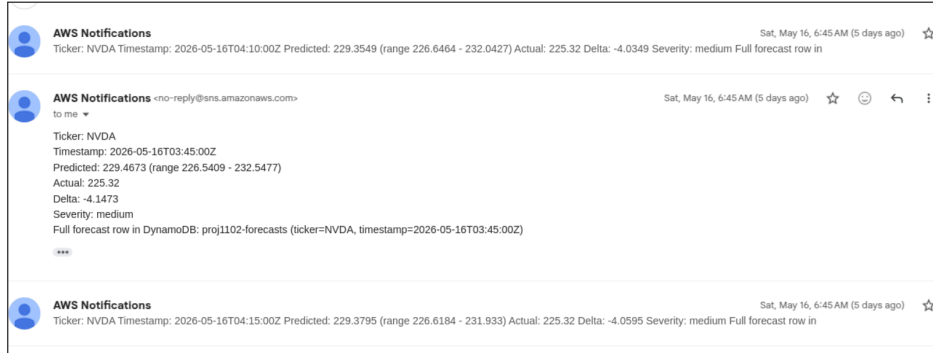
The cross-account wiring needs *both* sides: a resource policy on the table/stream in Ralph's account (148557232117) that grants Arthur's role access, plus an inline IAM policy on Arthur's side allowing **DescribeStream**, **GetRecords**, **GetShardIterator** and **ListStreams** on that stream. Once both sides match, the stream appears on the Lambda as a cross-account ("Intercompte") trigger in the Enabled state (Figure 7). Getting only one side right caused several of the issues in Section 12. SMS alerts were left out on purpose: SNS SMS is blocked while the account is in the default sandbox (it needs a manual AWS Support request and is not free), and email is free and enough for the team's monitoring. Source: lambda/anomalie_detection/.

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "dynamodb:DescribeStream",
8         "dynamodb:GetRecords",
9         "dynamodb:GetShardIterator",
10        "dynamodb:ListStreams"
11      ],
12      "Resource": "arn:aws:dynamodb:eu-west-1:148557232117:table/proj1102-forecasts/stream/*"
13    }
14  ]
15 }
```

(a) Arthur's inline IAM policy: the four DynamoDB-stream actions, scoped to Ralph's forecasts-table stream.



(b) The Lambda trigger: the cross-account ("Intercompte") DynamoDB stream, Enabled.



(c) A real anomaly alert delivered by SNS to email (NVDA, severity medium), with the full forecast row.

Figure 7: Anomaly alerting: the cross-account IAM policy and the enabled cross-account stream trigger that let Arthur's Lambda read Ralph's stream, plus the resulting email alert.

6.7 Orchestration - Step Functions

The two hourly Lambdas used to fire on independent EventBridge schedules, which meant the QuickSight refresh could run even if the forecast had errored. Wrapping them in a STANDARD Step Functions state machine (`proj1102-pipeline`) gave us:

- a **sequential dependency** (refresh only if the forecast succeeded);
- a **centralised failure path** (any uncaught error publishes one alarm email);
- **per-task retries** (2 attempts with backoff on transient errors);
- **explicit task timeouts**;
- a **dedicated CloudWatch log group** for execution events.

State-transition cost is negligible ($\approx \$0.000003/\text{day}$). Source: `stepfunctions/`.

6.8 Monitoring - CloudWatch and the freshness probe

Observability has three layers:

- **Lambda error alarms** (one per critical function);
- a **state-machine failure alarm**;
- a **custom data-freshness alarm**.

The freshness probe Lambda publishes `proj1102/oldest_parquet_age_minutes` every hour, and an alarm fires when any ticker's parquet drifts past a 2-hour budget. This is the alarm that catches the failure mode no Lambda-error alarm can see: *silent* ingestion death where nothing errors but data simply stops arriving. It caught a real partial NVDA outage in production (issue [12.4.1](#)). All alarms publish to the `proj1102-infra-alarms` SNS topic with the five team emails subscribed. Source: `lambda/freshness/`.

6.9 Self-improvement - weekly retraining

Two Lambdas keep the models from drifting without any human action: `proj1102-retrain` (Sunday 06:00 UTC) refreshes the hourly model and `proj1102-daily-retrain` (Sunday 07:00 UTC, staggered so SageMaker quotas do not collide) refreshes the daily model. Each inlines the corresponding training script's logic: `prep` $\rightarrow$ `submit training job` $\rightarrow$ `wait` $\rightarrow$ `create model + endpoint config` $\rightarrow$ `update the serverless endpoint in place` (blue/green, no downtime). Cost is $\approx$ \$0.02 per run. Source: `lambda/retrain/`, `lambda/daily_retrain/`.

7 The Strategic (Daily) Horizon - a Second Model

Because a 1-hour forecast, while operationally interesting, is not how a trading or risk team consumes forecasts (they look days-to-weeks ahead), we added a **second, independent DeepAR model** for the daily close.

Why not just extend the hourly model? The instinct is to bump the prediction length from 12 to ~ 2000 steps. That is a trap: a DeepAR trained on 5-minute bars has almost no skill past ~ 30 minutes (equity prices at that granularity are close to a random walk). Asking it to predict two weeks of 5-minute bars would produce bands so wide nothing is ever an anomaly, degrade the loss, and inflate every Step Functions run and DynamoDB write. The correct fix is a *different problem statement*: predict the daily close over the next few trading days using *years* of daily history.

Aspect	Hourly (operational)	Daily (strategic)
Frequency	5-minute bars	daily closes
Context window	120 bars (~ 10 h)	60 days (~ 12 weeks)
Prediction horizon	12 bars = 1 h	7 days
Data source	Alpha Vantage (via SQS)	Yahoo Finance v8 chart API
Inference cadence	hourly (Step Functions)	once per weekday, 20:15 UTC
Endpoint	<code>deepar-endpoint</code> (serverless)	<code>deepar-daily-endpoint</code> (serverless)
Anomaly alerts	✓ via Stream	✗ disabled by design
Feature use	sentiment as <code>dynamic_feat</code>	target-only (sentiment decays in hours)

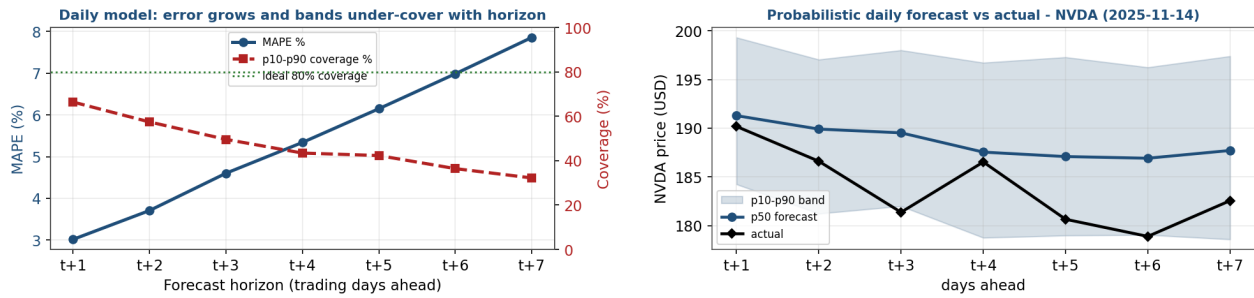
Table 8: Two complementary horizons, each calibrated to its own use case.

The daily model was bootstrapped from five years of daily closes (~ 1256 rows/ticker) pulled once with `yfinance`; the production Lambda then refreshes data through Yahoo's public chart API directly via `urllib`, avoiding the need to package `yfinance` in the deployment zip. Anomaly alerts are deliberately *off* for this horizon: a single daily close outside the band is too noisy to be actionable, and (as the backtest below shows) at the daily horizon a band breach is usually the band being too narrow rather than a genuine anomaly.

Walk-forward backtest. We ran a rolling backtest over ~ 6 months (357 test windows, 2499 per-step predictions). Two findings are worth reporting honestly:

Horizon	t+1	t+2	t+3	t+4	t+5	t+6	t+7
MAPE %	3.0	3.7	4.6	5.3	6.1	7.0	7.9
p_{10}-p_{90} coverage %	66	57	50	43	42	36	32

Table 9: Daily-model backtest. MAPE grows sub-linearly with horizon ($\sim 3\%$ at $t+1$ to $\sim 8\%$ at $t+7$), the expected accumulation of uncertainty; meanwhile the 80% interval *under-covers*, i.e. DeepAR reports tighter bands than its error supports on volatile equities, which is precisely why anomaly alerting is kept on the hourly model only.



(a) Error rises and coverage falls as the horizon lengthens.

(b) A real 7-day forecast window: actual inside the p_{10} - p_{90} band.

Figure 8: Daily-model walk-forward backtest, plotted from `sagemaker/backtest_*.csv`.

Honesty caveat (stated in the report on purpose). This backtest is *in-sample*: the deployed daily model was trained on data that includes the test points, so these numbers read as “the model fit this history well”, not “it will perform this well on unseen data”. A true out-of-sample walk-forward (retrain at each window) is listed as future work; it would cost a few dollars in incremental training and likely push MAPE up 20-30%.

The result is **two horizons, two models, one shared dashboard**: an operational layer that catches near-real-time band breaches and feeds alerts, and a strategic layer that gives a forward-looking weekly outlook - addressing the days-to-weeks consumption pattern while preserving the original near-real-time scope intact.

8 Visualisation: the QuickSight Dashboard - Francesco

The visualisation layer turns raw forecast outputs into interactive analytics. Two S3 buckets feed it - one holds actual market values + sentiment (from Glue + Comprehend), the other the SageMaker forecast predictions. **Amazon Athena** provides a unified query layer (an EXTERNAL table per source plus a UNION ALL view that puts the past actual line and the future band on one timeline), and **QuickSight** (Direct Query) renders it. QuickSight would not run on Francesco’s own AWS account (a persistent account-level error that we could not resolve), so the two of us moved the visualisation layer into Ralph’s account and worked on it together: **Ralph** enabled QuickSight there, provisioned the Athena database, the EXTERNAL tables and the UNION ALL view, and wired the hourly refresh Lambda; **Francesco** then designed the sheets, charts, filters and interactions on top of it. Athena hit the same account limitation on Francesco’s side, so Ralph also set up its configuration following Francesco’s guidance. It is a concrete example of the cross-account pair-work the project repeatedly demanded: one person owns the design, another owns the plumbing in the account that can actually run the service.

The dashboard is split into two complementary sheets.

8.1 Hourly Forecasts sheet (operational)

Short-range, 1-hour-ahead view updated at 5-minute granularity, for near-real-time monitoring. For each ticker it shows the **actual** price, the p_{50} **median forecast**, and the p_{10}/p_{90} **confidence band**, plus five analytical panels: a **spread comparison** (band width $p_{90} - p_{10}$ = uncertainty per ticker), **forecast-bias** KPIs (latest actual vs. p_{50} : over/under-estimation), **MAPE per ticker** (comparable across price levels), an **anomaly counter** (how many actuals fell outside the band), and a **latest-actual-value table** that lists, per ticker, the most recent observed price and its timestamp. A dynamic control lets the user switch the window between 6 h, 12 h and 24 h.

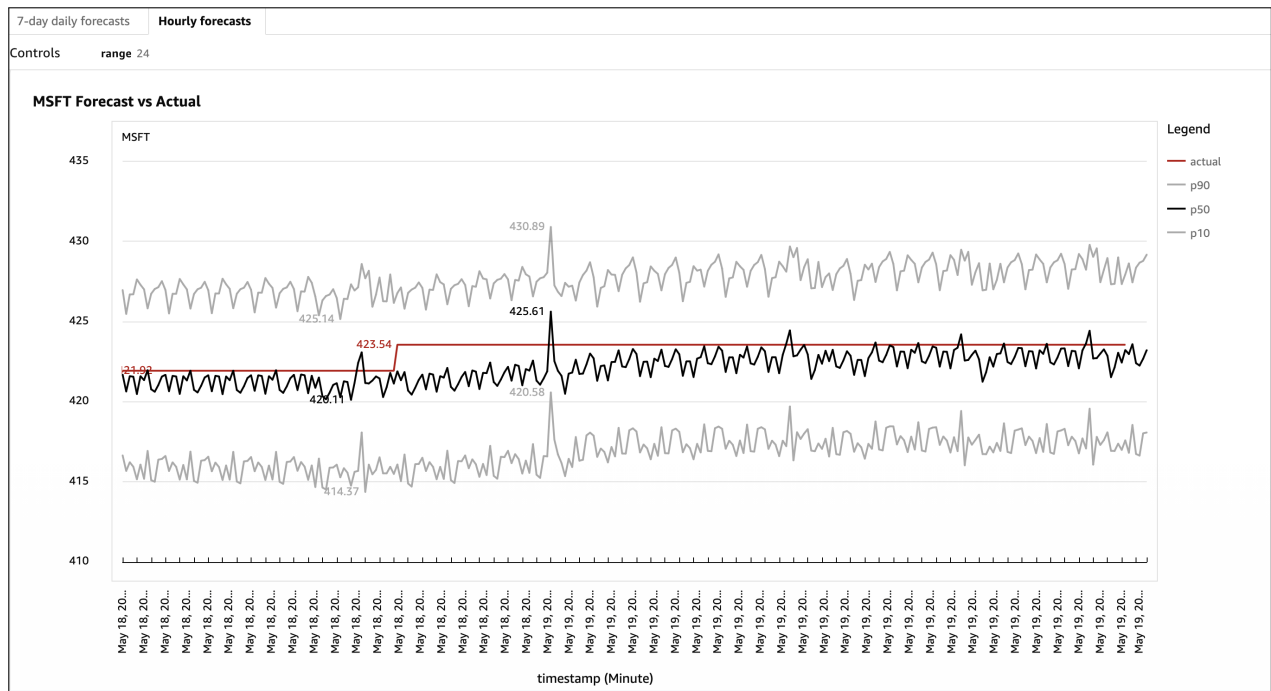


Figure 9: Hourly Forecasts sheet, MSFT: actual vs. $p_{10}/p_{50}/p_{90}$ over the last 24 h. The black p_{50} line tracks the red actual closely, inside the grey band.

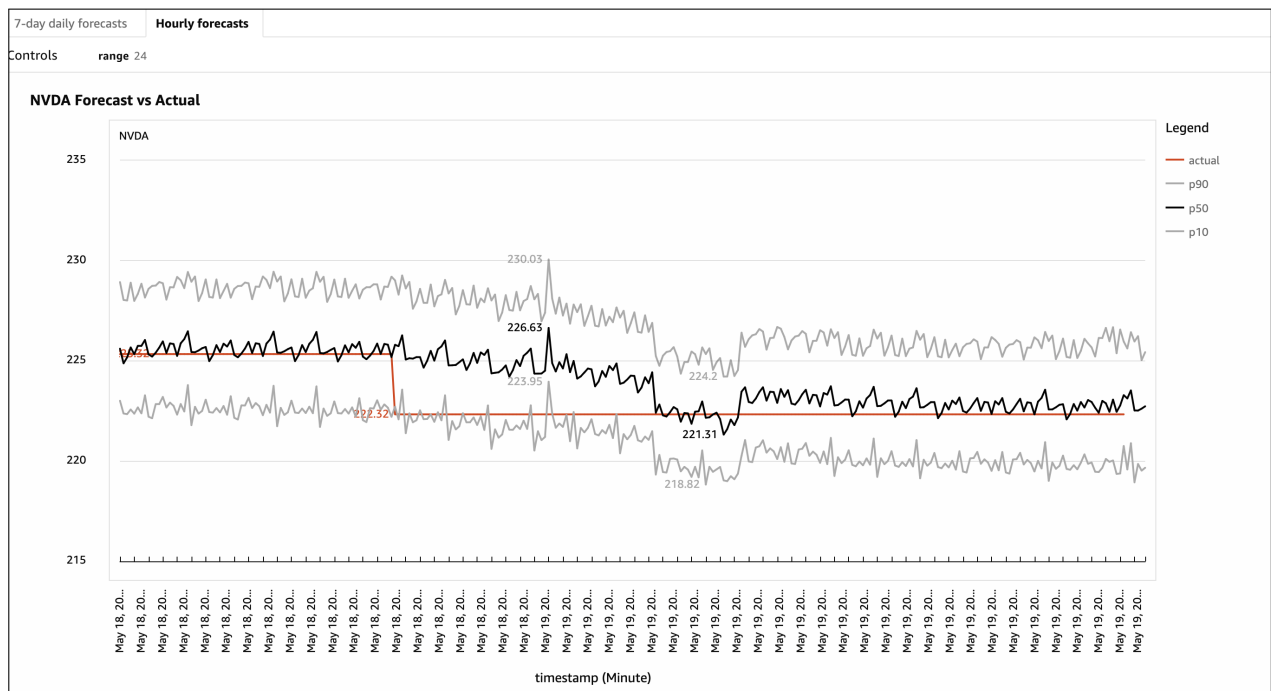


Figure 10: Hourly Forecasts sheet, NVDA: actual vs. the p_{10} - p_{90} band, last 24 h.

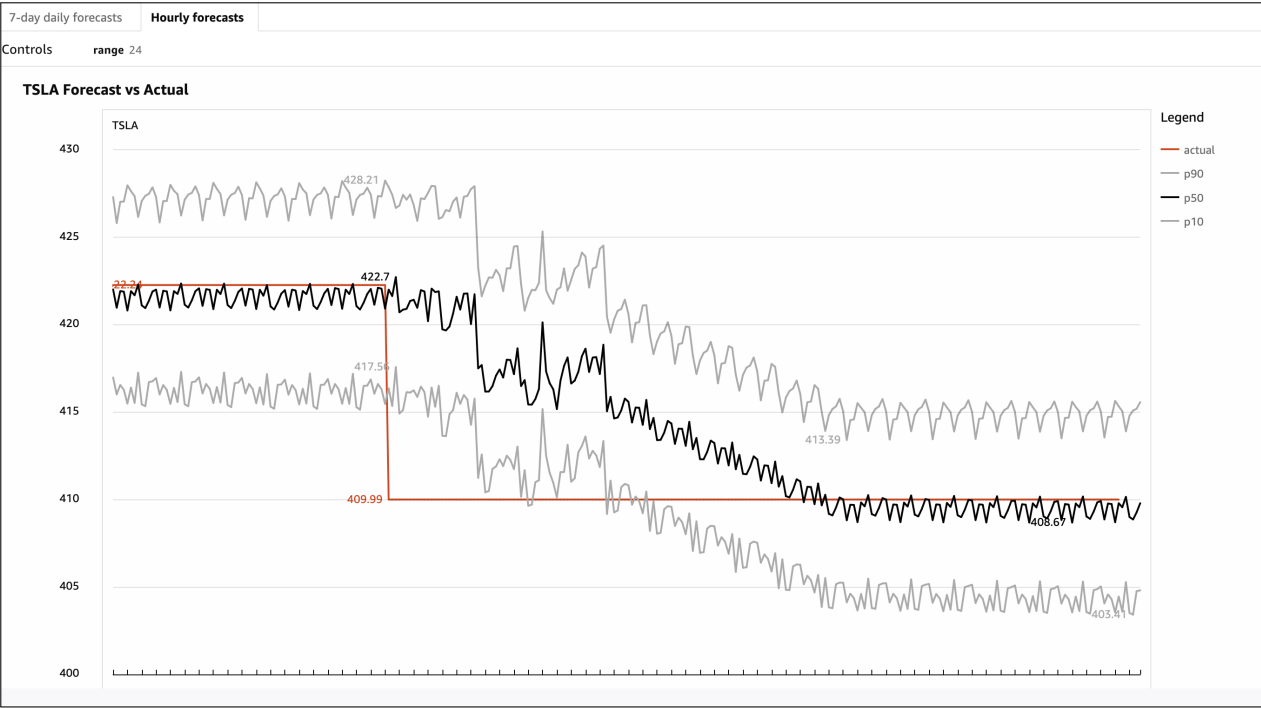
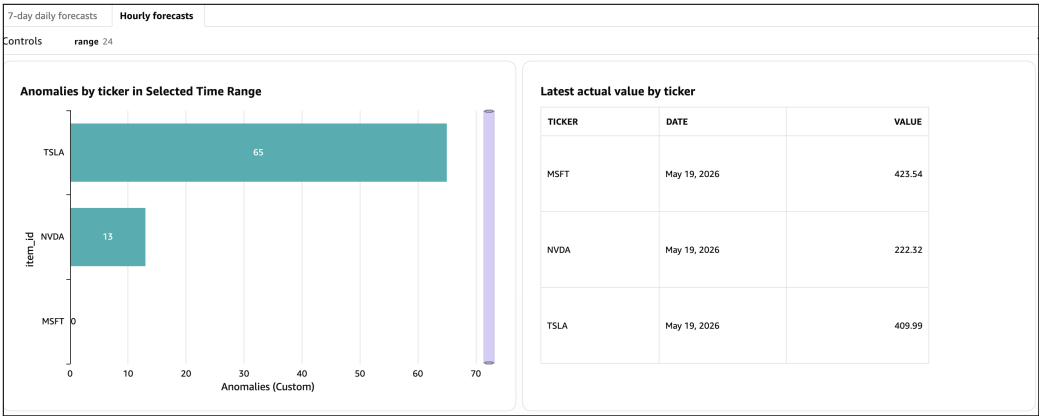


Figure 11: Hourly Forecasts sheet, TSLA: actual vs. the p_{10} - p_{90} band, last 24 h.



(a) Anomaly counter + latest actual value per ticker.



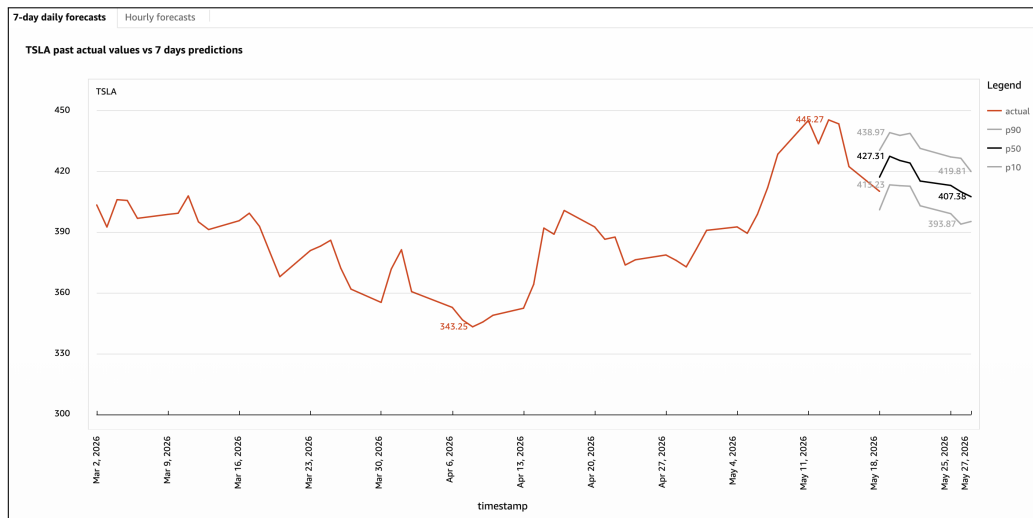
(b) Forecast-bias KPIs (actual vs. p_{50}).

(c) MAPE (%) per ticker.

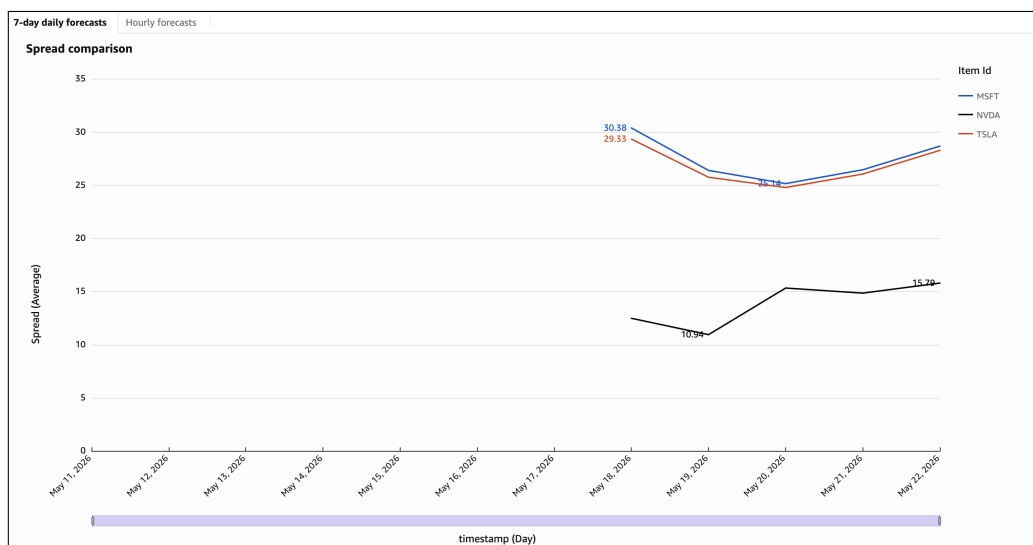
Figure 12: Hourly Forecasts sheet - the analytical panels (anomaly counter, forecast-bias KPIs, per-ticker MAPE), all driven by the 6/12/24 h time-range control.

8.2 7-day Daily Forecasts sheet (strategic)

Longer horizon: 7 days ahead against up to three months of history, for medium-term trend and uncertainty analysis, with a spread-comparison chart across the three assets.



(a) TSLA: 3 months of actuals + 7-day forecast band.



(b) Spread comparison ($p_{90} - p_{10}$) across TSLA / MSFT / NVDA.

Figure 13: The 7-day Daily Forecasts sheet: strategic, longer-horizon view.

8.3 Interactive features

Three interactions make the dashboard explorable rather than static:

- a **time-range filter** (6/12/24h) on the operational sheet;
- a **navigation action** that switches between the short- and long-term views for a ticker by clicking a point on a chart;
- a **URL action**: clicking a ticker value opens that asset's Yahoo Finance page (the URL embeds the ticker, e.g. finance.yahoo.com/quote/TSLA) for deeper context.

The live dashboard URL is included in the team's submission.

9 Services and Resources: Benefits and Alternatives

The brief asks, for each service used, why the project benefits from it and what alternative was discarded and why. Table 10 answers this for every AWS service in the system. The unifying theme is **serverless, pay-per-use, scale-to-zero** - appropriate for a bursty, low-volume student pipeline that must cost almost nothing yet stay always-on.

Service	Why we benefit	Alternative discarded
AWS Lambda	Zero idle cost, event-driven, free tier covers our volume; one function per concern.	EC2/containers - always-on cost, patching, over-kill for sub-second tasks.
EventBridge	Managed cron with no server; native Lambda/Step Functions targets.	A cron on an EC2 box - a single point of failure to maintain.
Amazon SQS	Decouples ingestion from processing, buffers bursts, gives retries + dead-letter queues.	Direct Lambda-to-Lambda calls - tight coupling, no buffering, lost messages on failure.
AWS Glue (Python Shell)	Serverless Spark/pandas ETL; scales per run; no cluster to manage.	A long-running EMR cluster - expensive and idle most of the hour.
Amazon S3	Cheap, durable shared data plane across four accounts; Parquet-friendly.	EBS/EFS - not cross-account, not object-addressable.
Amazon Comprehend	Managed sentiment, free under 10M chars/mo, no model to package or train.	A bundled HuggingFace model - large zip, cold-start cost, maintenance.
SageMaker DeepAR (serverless)	Probabilistic multi-series forecaster with native related-time-series ($p_{10}/p_{50}/p_{90}$); serverless = \$0 idle.	Real-time endpoint (~\$43/mo idle); AWS Forecast (deprecated); Prophet/ARIMA (no shared global model, weaker intervals).
Amazon DynamoDB	Single-digit-ms key-value store; Streams give us a free event source for alerts; PAY_PER_REQUEST.	RDS - always-on instance, no native stream-to-Lambda, heavier for a key-value access pattern.
Amazon Athena	Serverless SQL straight over S3 CSV/Parquet; pay-per-query; trivial to refresh.	Loading into Redshift - cluster cost and ETL overhead for a small dataset.
Amazon QuickSight	Managed BI with Direct Query (no SPICE capacity needed) and rich interactions.	Self-hosted Grafana/Streamlit - a server to run and secure.
Amazon SNS	Fan-out email alerts, cross-account topic policies, free at our volume.	Custom SMTP/SES wiring - more code, no built-in subscription management.
AWS Step Functions	Declarative orchestration: ordering, retries, timeouts, one failure path, execution logs.	Chained EventBridge schedules - no dependency guarantee, no central error handling.
CloudWatch	Logs + metrics + alarms with zero setup; custom metric for data freshness.	A third-party APM - cost and integration effort for no added value here.

Table 10: Every AWS service used, the concrete benefit, and the main alternative we discarded.

Design principles. The architecture deliberately satisfies the five principles the brief lists:

- *design for failure*: SQS + DLQs, Step Functions retries/catch, freshness alarm, weekly retrain;
- *security*: no secrets in code, least-privilege cross-account resource policies, per-owner accounts;
- *elasticity*: Lambda concurrency, SageMaker serverless **MaxConcurrency**, DynamoDB on-demand;
- *decoupling*: queues, S3, and DynamoDB Streams between every stage;
- *performance*: Parquet, Direct Query, 10 h context windows tuned to the horizon.

10 Twelve-Factor Methodology

The brief requires an explicit, point-by-point discussion of the twelve-factor methodology, including honest justification where a factor is not fully met. Every claim below maps to a concrete file or

AWS resource that can be verified in the repository.

#	Factor	State	Evidence
1	Codebase	✓	One monorepo versions every component; each push lands a deployable revision.
2	Dependencies	✓	Per-Lambda <code>requirements.txt</code> ; pandas/pyarrow/numpy from the AWS-managed layer (named in every <code>deploy.sh</code>); dev/test deps in <code>pyproject.toml</code> .
3	Config	✓	No secrets/env-specific values in code; every tunable read from a Lambda env var; <code>.env.example</code> checked in.
4	Backing services	✓	S3, SQS, DynamoDB, endpoints, SNS, Athena are all attached resources referenced by ARN/URL via env - swap account/region by editing env, not code.
5	Build/release/run	✓	<i>Build</i> = GitHub Actions CI (compile + lint + ASL validation + pytest + Docker build); <i>Release</i> = idempotent <code>deploy.sh</code> per component, one-shot <code>infrastructure/setup_phase*.py</code> scripts that provision the Glue job and Lambdas end-to-end, and a root Makefile; <i>Run</i> = EventBridge/Step Functions, never a human.
6	Processes	✓	Stateless Lambdas + stateless Glue job + serverless endpoints; all durable state in S3/DynamoDB/model artifact.
7	Port binding	○	N/A by design - event-driven pipeline, not an HTTP service; triggers are SQS, EventBridge, DynamoDB Streams.
8	Concurrency	✓	Horizontal scale everywhere: Lambda per-invocation, SageMaker serverless <code>MaxConcurrency=5</code> , DynamoDB on-demand.
9	Disposability	✓	1-3s cold start; SQS/Stream event-source mappings re-drive failed batches; endpoints scale to zero; Step Functions retries absorb transients.
10	Dev/prod parity	○	Single production environment (deliberate scope choice). Because every name is config (factor 3), a second environment is a config change, not a code change - but we did not pay to operate two parallel pipelines.
11	Logs	✓	Every Lambda streams <code>print()</code> to CloudWatch; Step Functions events to a dedicated log group; custom freshness metric; documented Logs Insights queries (<code>docs/cloudwatch_insights.md</code>).
12	Admin processes	✓	One-off admin entry points: <code>housekeeping/check_pipeline.py</code> (live health board, also <i>containerised</i>), <code>scripts/backtest_daily.py</code> ; all invoked via the Makefile.

Net result: 10 factors fully satisfied, 1 N/A by design (port binding - we are not an HTTP service), 1 partial (dev/prod parity - a single production environment was a conscious scope decision). The partial gap is small precisely because factor 3 (config from env) makes a second environment a configuration change rather than a code change.

Twelve-factor compliance

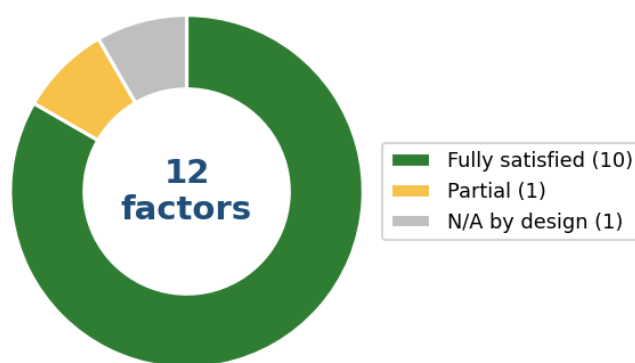


Figure 14: Twelve-factor compliance at a glance: 10 fully satisfied, 1 partial (dev/prod parity), 1 N/A by design (port binding).

11 Code Organisation and Engineering Practices

11.1 Repository structure

The code lives in one private GitHub monorepo, grouped by component so each piece is independently deployable and easy to review:

Path	Contents
<code>config/</code>	pipeline-wide config (assets, queue URLs)
<code>infrastructure/</code>	one-shot provisioning scripts (<code>setup_phase1.py</code> , <code>setup_phase2.py</code>) that stand up the Glue job and Lambdas
<code>glue/data_preparation/</code>	Dalibor's Glue Python Shell job
<code>lambda/</code>	one folder per Lambda (ingest, sentiment, glue_trigger, forecast, retrain, freshness, daily_ingest, forecast_daily, daily_retrain, anomalie_detection)
<code>sagemaker/</code>	DeepAR train + deploy scripts (hourly + daily)
<code>stepfunctions/</code>	orchestration state machines (ASL JSON + deploy)
<code>quicksight/</code>	Athena DDL + refresh Lambda
<code>housekeeping/</code>	live health probe + Dockerfile
<code>tests/</code>	pytest unit tests + conftest
<code>docs/</code>	CloudWatch Insights queries + <code>team/</code> handoff records
<code>.github/workflows/</code>	CI pipeline; the root <code>Makefile</code> gives single-entry commands

Table 12: Repository layout - one private GitHub monorepo, grouped by component so each piece is independently deployable.

11.2 From the labs to the project

The brief asks us to reuse the patterns from the course laboratories, so this subsection maps each relevant lab to where the project applies it. It is a requirements-coverage map; the detailed explanation of how each component actually works is in Section 6.

Lab	How proj1102 applies it
Lab 4 - CI/CD	GitHub Actions workflow (<code>.github/workflows/ci.yml</code>) runs on every push: compiles all Python, lints every <code>deploy.sh</code> , validates Step Functions JSON, runs <code>pytest</code> , builds the Docker image. A <code>housekeeping/check_pipeline.py</code> admin probe mirrors lab 4's <code>admin_script.py</code> .
Lab 5 - Observability & testing	<code>pyproject.toml</code> + a <code>tests/</code> suite of 50 unit tests over the pure helper functions of the forecast, Glue ETL and ingestion components; <code>confest.py</code> pre-sets env vars before import; <code>print()</code> captured as CloudWatch events.
Lab 6 - Serverless	The entire pipeline: 11 Lambdas, 2 serverless endpoints, 2 on-demand DynamoDB tables, SQS, EventBridge, Step Functions - no EC2, no always-on instance.
Lab 7 - Log analysis	Six documented CloudWatch Logs Insights queries (<code>docs/cloudwatch_insights.md</code>); two of them pinned down real incidents (Section 12).
Lab 8 - Extracting information	Comprehend sentiment, DeepAR forecasting on two horizons, and a walk-forward backtest.

Table 13: Mapping the course labs onto the project.

11.3 Docker and CI

Although the production system is serverless and does not itself need containers, the lab 4 pattern asks for containerisation. The natural fit is the **housekeeping health probe**: a stateless command-line tool that checks every Lambda, endpoint, alarm, table and parquet age. It is containerised (`housekeeping/Dockerfile`, non-root user, pinned to the same library major versions as the Lambda layer) so it runs identically on a laptop or a CI runner via `make docker-run`. The CI workflow builds that image on every push, so the container never rots.

12 Problems Encountered and Solutions

A real distributed system over four accounts produced real bugs. We logged every one with a symptom, a root cause, a resolution and a lesson; the full ledger lives in `REPORT_NOTES.md`. Below we deep-dive the four most instructive, then summarise the rest. They cluster into three families the brief asks about: **coding / data contracts, services and cross-account resources**, and **team organisation**.

12.1 Coding and data-contract problems

12.1.1 The forecast was predicting the past

Symptom. A 15:08 run wrote predictions for the 13:30-14:25 window - already fully elapsed before the run started.

Cause. The forecast cron (:08) fired *before* Glue (:29), so it always read the previous cycle's stale parquet; DeepAR anchors predictions to the last timestamp in the data.

Fix. Move the forecast to :45 and the refresh to :55 - strictly after Glue - and match the horizon (1h) to the cadence (1h).

Lesson: schedule the consumer after the producer; an obvious ordering ran wrong in production for days.

12.1.2 The closed-market anomaly storm

Symptom. ~300 alert emails between Friday's US close and Saturday morning; NVDA's anomaly rate jumped to 37%.

Cause. When the market closes the feed pins `actual` at the last trade, while the model keeps extrapolating Friday's upward trajectory - so every backfilled row reads `actual < p10` and fires.

Fix. A `_is_market_open(ts)` gate computes the anomaly flag only during 13:30-20:00 UTC Mon-Fri; outside that, the actual is still recorded faithfully but no alert fires. Post-fix market-hours anomaly rate: 0%.

Lesson: the statistical definition ("outside the band") and the operational one ("outside the band *during trading hours*") diverge whenever the feed produces semantically meaningless values.

Two further coding bugs were pure data-contract mismatches: predictions never accumulated in QuickSight because the forecast CSV was keyed by *date* (every hourly run overwrote the same file - fixed by adding the run timestamp to the filename); and sentiment never reached the model because Glue joined on an *exact* 5-minute timestamp (news rarely lands on a slot - fixed with a backward `merge_asof`, lifting coverage from <1% to >87%).

12.2 Services and cross-account problems

12.2.1 DynamoDB rejected the stream resource policy

Symptom. `PutResourcePolicy` failed with "Invalid action names" when granting stream actions on the table.

Cause. DynamoDB requires *separate* policies for the table and the stream; stream actions (`DescribeStream`, `GetRecords`, `GetShardIterator`) are not valid in a table policy.

Fix. Attach two policies - table actions on the table ARN, stream actions on the stream ARN. A follow-on bug granted the wrong *account* (team sandbox instead of Arthur's personal account).

Lesson: every cross-account boundary has *two* IAM surfaces (the producer's resource policy and the consumer's role) and they fail with the same opaque "AccessDenied".

The subtlest cross-account bug: Dalibor's Glue *alarm* could not publish to Ralph's SNS topic even though the team *account* was granted publish - because CloudWatch alarms publish as the `cloudwatch.amazonaws.com service principal`, not as a user. The fix added a `service-principal` statement with a `SourceAccount` condition.

Lesson: cross-account integration has *three* layers - the consumer role, the producer resource policy, and *which principal type actually performs the action*.

Other service issues: QuickSight "cannot publish" (dataset was in SPICE mode with zero SPICE capacity - switched to Direct Query), and email alerts arriving as raw JSON (the subscription was Email-JSON, not Email).

12.3 Team-organisation problems

The recurring organisational lesson is that *a component that runs cleanly in isolation can still be silently broken at its interface*. The clearest case: Ralph's sentiment Lambda suddenly stopped scoring anything, because *no news was arriving*. Tracing the empty news queue upstream revealed that Ilias's "deployed" ingestion Lambda in the team account was a dead Phase-1 leftover (no log group, a stale asset list dated weeks earlier), while the real feed had been redeployed to his personal account and was no longer writing to the queue Ralph's Lambda consumed. The sentiment branch was starved without a single error being raised anywhere. Separately, NVDA was flagged anomalous 35% of the time because the deployed model had been trained before NVDA was added to the asset set. Both trace back to the coordination rule we sometimes broke: *announce every schema or deployment change to the group*. These are exactly why we wrote the per-pair handoff contracts (Section 4).

12.4 What the monitoring layer caught

12.4.1 A real partial ingestion outage

Event. The freshness alarm fired Saturday 13:36 UTC - NVDA's parquet was stuck 136 min while TSLA/MSFT were fresh.

Diagnosis. NVDA *news* kept flowing (sentiment scored NVDA at 13:25) but NVDA *prices* stopped - the asymmetric lag across tickers ruled out a market-closed cause and pointed to NVDA's Alpha Vantage price keys exhausting.

Why it matters. This was a genuine production incident, caught with no human polling, through the full path Lambda → custom metric → alarm → SNS → email. It is the failure mode no Lambda-error alarm can see: nothing errored, data just stopped.

The same alarm family produced one instructive *false* positive: the daily-parquet alarm fired every Monday because the threshold (80 h) didn't account for **yfinance** dating Friday's close at 00:00 UTC - a 20 h offset - making the natural weekend gap ~92 h. Re-calibrated to 120 h.

Lesson: alarm thresholds must match the data's actual timestamp convention, not human intuition about when the event "happened".

Problem	Root cause	Resolution
Stale ingestion Lambda	live code deployed to personal acct; team-acct copy dead	confirmed live feed; need deployed-vs-git check
Predictions frozen in QuickSight	CSV keyed by date, runs overwrote	add run-timestamp to filename + dedup keep-last
Forecasting the past	consumer scheduled before producer	reschedule :45/:55 after Glue
Sentiment never joined	exact 5-min join, news is sparse	backward <code>merge_asof</code> , 4h tolerance
NVDA 35% anomalies	model trained before NVDA added	retrain on live 3-ticker set
QuickSight publish fails	SPICE mode, 0 SPICE capacity	switch to Direct Query
Anomaly Lambda crash on INSERT	read ["BOOL"] on a NULL field	filter MODIFY, guard the key
DynamoDB policy rejected	stream actions need separate policy	two policies (table + stream)
Stream policy wrong account	granted sandbox, not Arthur's acct	re-grant Arthur's personal account
Alerts as raw JSON	Email-JSON subscription	re-subscribe with Email protocol
SFN deploy: policy quota	10 managed policies already attached	use an inline policy
Alarm can't publish x-acct	granted user, not service principal	add <code>cloudwatch.amazonaws.com</code> statement
Closed-market storm	feed pins price at close	market-hours gate on anomaly flag
Freshness false positive (Mon)	threshold ignored yfinance 00:00 stamp	raise threshold to 120h
CI failed first run	layer-provided deps absent on runner	install boto3/pandas/pyarrow in CI

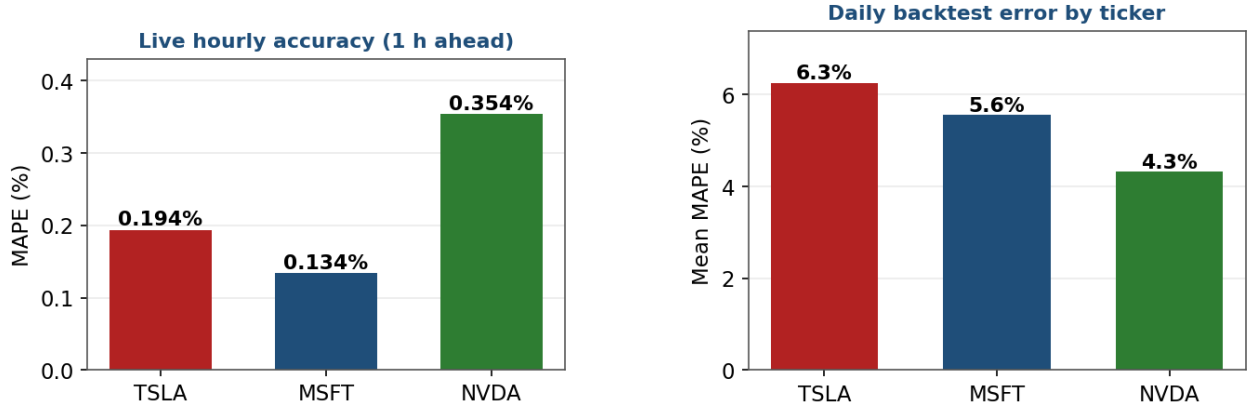
Table 14: The full issues ledger (condensed). Each is documented at length in `REPORT_NOTES.md`.

13 Results and Live Accuracy

Because the system runs continuously, accuracy is measured on *real* backfilled predictions, not a held-out file. After retraining on the live three-ticker set (and adding sentiment as a feature), the operational model is highly accurate at the 1-hour horizon, and after the market-hours fix the anomaly rate is zero across all tickers during trading hours.

Ticker	Rows backfilled	MAPE	Anomalies (market hrs)
TSLA	528	0.194%	0%
MSFT	432	0.134%	0%
NVDA	432	0.354%	0%

Table 15: Live hourly-model accuracy from backfilled DynamoDB rows (market-hours timestamps only). NVDA was 1.36% MAPE / 35% anomalies before retraining on the correct asset set.



(a) Live 1-hour-ahead MAPE per ticker - all well under 0.4%.

(b) Daily-model backtest MAPE per ticker (6-month rolling).

Figure 15: Accuracy by ticker for both horizons (generated from the live DynamoDB history and the backtest CSV).

Training quality improved measurably as the dataset and features were corrected: test wQL fell from 0.005 (mixed/early asset set) to **0.0029** after retraining on TSLA/MSFT/NVDA with sentiment as a `dynamic_feat`. The dashboard figures in Section 8 are live evidence: the actual line tracks the p_{50} median closely, and the bias KPIs sit within $\pm 0.5\%$. The daily model's backtest (Section 7) shows the expected horizon-growing error and an honestly under-covering band - the reason daily alerts are disabled.

14 Cost Analysis

The single biggest cost trap on a SageMaker project is a real-time endpoint left running ($\sim \$43/\text{month}$ for `ml.t2.medium`). Choosing **serverless inference** - which scales to zero between calls - removed that trap entirely: $\approx \$0$ idle, ≈ 1 s of compute per cycle. Everything else stays inside the AWS free tier or on-demand cents.

Component	Mode	Cost
Both DeepAR endpoints	serverless	\$0 idle, ~ 1 s per call
11 Lambdas (hourly + daily + ops)	invocation	free tier (1M/mo)
DynamoDB (both tables)	PAY_PER_REQUEST	cents/month
Comprehend DetectSentiment	free tier	\$0 (< 10 M chars/mo)
Step Functions STANDARD	per transition	$\sim \$0.000003/\text{day}$
Training (hourly + daily)	on-demand	$\sim \$0.04/\text{week}$ ($\sim \$2/\text{yr}$)
AWS Glue (Python Shell)	per run	$\sim \$0.15/\text{month}$
Amazon S3 (Parquet + raw)	storage	$\sim \$0.05/\text{month}$ (grows with Parquet size)

Table 16: Per-component cost. Cumulative spend $\approx \$0.35$ of a \$100 academic credit; steady-state $\approx \$0.20/\text{month}$ (Glue + S3; all compute scales to zero).

15 Hours Invested and Deviation

The brief budgets ≈ 40 hours per member (≈ 200 h for the team of five) and asks for the breakdown by task category, the deviation from the plan, and how we could have deviated less. Table 17 reports our effort by category.

Member	Design	Research	Coding	Meetings	Docs	Total
Ilias El Mariky	4	7	22	6	5	44
Dalibor Švonavec	4	6	25	6	5	46
Ralph Khairallah	5	7	27	6	5	50
Francesco Barillari	4	7	23	6	6	46
Arthur Bohin	5	6	21	6	5	43
Team total	22	33	118	30	26	229

Table 17: Hours by member and task category. Dalibor’s coding figure (~ 25 h) is self-reported in the team document; Ralph’s slightly higher total reflects the cross-account infrastructure work and the unplanned second (daily) horizon.

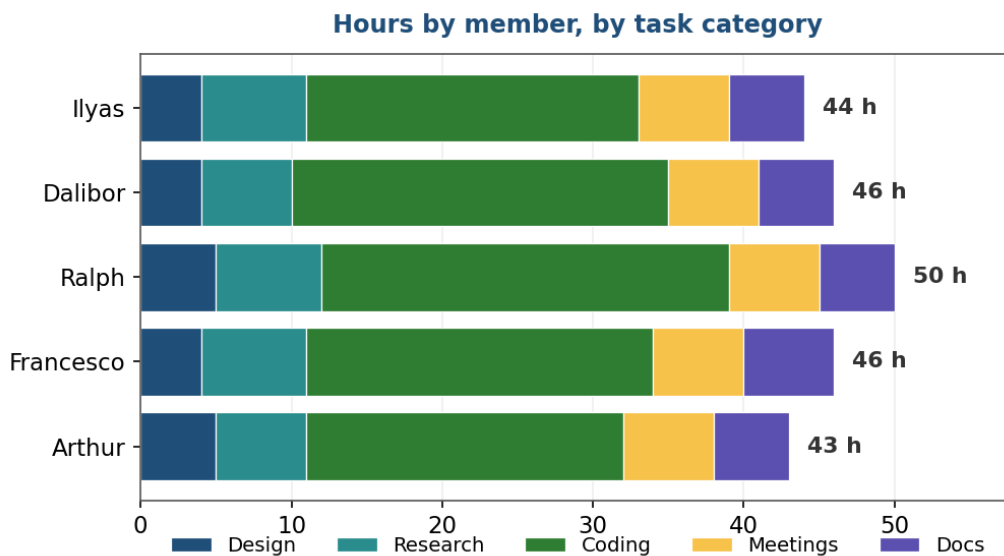


Figure 16: Hours per member, broken down by task category. Coding/integration dominates; Ralph’s bar is a little longer (cross-account infrastructure + the unplanned daily horizon).

Note on figures

Per-member totals are self-reported by each member against their own work logs. *Coding* here counts all hands-on build work, including configuration done directly in the AWS console (IAM, Glue, SageMaker, QuickSight, DynamoDB), not only code committed to the repository - which is why a member who worked mostly in the console still shows substantial coding effort.

Deviation. The team total (~ 229 h) is roughly **+15%** over the 200 h baseline. The overrun is concentrated in *coding/integration*, and almost all of it is explained by two things the first draft under-estimated:

- **cross-account IAM:** every integration boundary needed a resource policy *and* a consumer role, and the failure mode is always an opaque “AccessDenied”, which is slow to diagnose (Section 12);
- the **second (daily) forecasting horizon**, which was not in the original plan and added a model, two Lambdas, a state machine and a backtest.

Ralph’s slightly higher total reflects absorbing the cross-account plumbing and the unplanned daily horizon.

How we could have deviated less. A shared cross-account IAM template (or CloudFormation/CDK stack) from the start would have removed most of the repeated resource-policy debugging. Agreeing the asset set and DynamoDB schema *once* and freezing them earlier would have avoided the retrain-on-wrong-tickers and join-key reworks. And a “signal flows end-to-end” smoke test per

handoff would have caught the silent-interface bugs in minutes rather than days.

16 Conclusions and Future Work

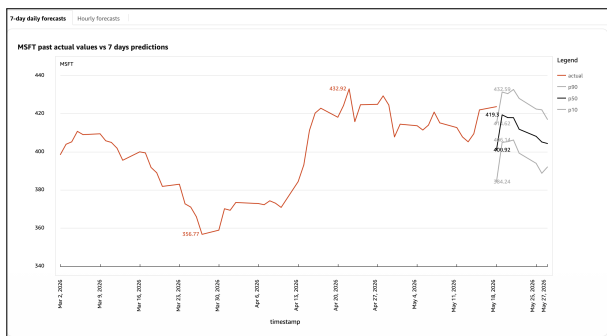
The project delivers what the first draft set out to do and more: a **fully automatic, near real-time financial forecasting pipeline** on AWS that ingests live data, scores sentiment, forecasts two horizons with serverless DeepAR, visualises everything on a live interactive dashboard, and alerts on real anomalies - all for about \$0.20/month and across four cooperating AWS accounts. It satisfies the design principles the brief asks for (failure-tolerant, secure, elastic, decoupled, performant), follows the twelve-factor methodology (10/12 fully), and applies the CI/CD, observability, serverless and log-analysis patterns from the labs.

The most valuable lessons were organisational as much as technical: in a multi-account, multi-owner system the *interfaces* are where things break, so explicit handoff contracts and a single shared naming/region convention paid for themselves many times over. The monitoring layer proved its worth by catching a genuine production outage with no human in the loop.

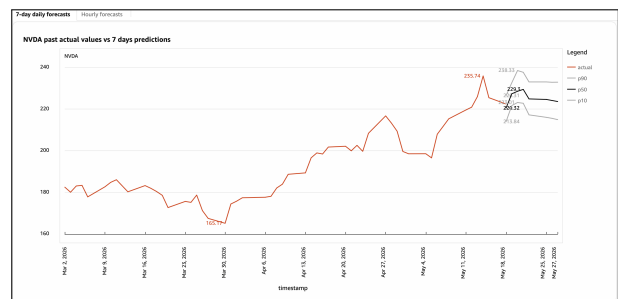
Future work.

- A true out-of-sample walk-forward backtest (retrain at each window) for an honest daily-horizon accuracy number.
- A *streak*-based anomaly rule for the daily horizon (e.g. three consecutive closes below p_{10}) for genuine regime-change detection.
- Replacing the bash deploy scripts with CloudFormation/CDK for full, reproducible cross-account provisioning.
- A second (staging) environment, now cheap to add because every resource is configured through environment variables.

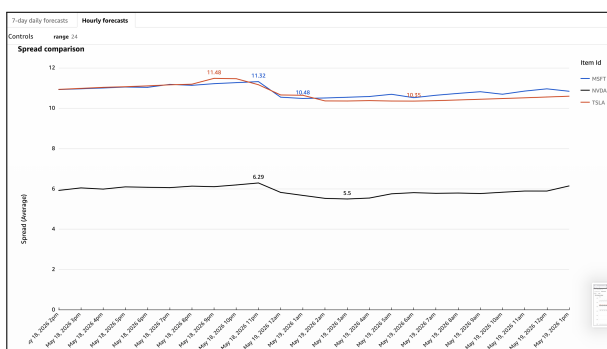
Annex A - Additional Dashboard & Console Evidence



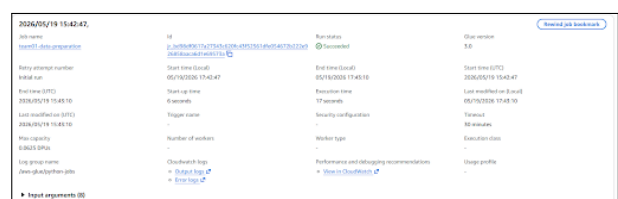
(a) Daily: MSFT, 3 mo actual + 7-day band.



(b) Daily: NVDA, 3 mo actual + 7-day band.



(c) Hourly: prediction-band spread per ticker.



(d) A single successful Glue run detail.

Figure 17: Further evidence from the live dashboard and the Glue console.

Annex B - Reproducibility and Team Documents

Every component is deployed by an idempotent script wired into a single **Makefile**; common operations are one command each:

Command	What it does
<code>make check</code>	static checks (Python compile + bash + ASL JSON)
<code>make test</code>	pytest unit suite
<code>make health</code>	live AWS health probe (all Lambdas / endpoints / alarms)
<code>make docker-run</code>	the same health probe inside Docker
<code>make deploy-hourly</code>	sentiment + forecast + retrain + freshness
<code>make deploy-daily</code>	daily ingest + forecast + retrain

Table 18: Common operations - every component's deploy is wired into a single **Makefile**.

The team's coordination artefacts are versioned alongside the code in **docs/team/**: the *Team Coordination Plan* (task split, conventions, weekly order of work) and the four pairwise *handoff* documents (Ilias→Dalibor, Dalibor→Ralph, Ralph→Francesco, Francesco→Arthur). The full per-issue engineering ledger is in **REPORT_NOTES.md** and the running cost record in **COST_LOG.md**.